

Flight Board · Project Specification

Basis for building the project: a departure board with real data from Prague airport on GKE. Version 8 (English), 9 Oct 2026. Specification for starting implementation. Where a brief description in a main chapter differs from an implementation rule in Appendix A, Appendix A prevails.

1. Project goal

Build a mini version of an airport departure board (FIDS – Flight Information Display System) with real departure data from Prague. Infrastructure is created with Terraform on GCP; the application is deployed to GKE with GitHub Actions. The code is written by two AI agents (Codex and Claude Code) using pull requests and mutual review; the author assigns tasks and approves merges. The focus is a reliable data flow: the board never shows corrupted data and always says clearly when data may not be up to date.

Area	Decision
Cloud	GCP, zonal GKE in europe-west3-a, 3 standard e2-standard-2 nodes (not spot).
Application	One Spring Boot application (Java 25, Spring Boot 3.5): coordinated fetch, validation, API. Frontend React + TypeScript served by nginx.
Database	MongoDB replica set with 3 members via a ready-made Helm chart. Locally a single-member replica set (for transactions).
Data source	AeroDataBox FIDS, PRG departures, 30-minute interval (configurable), Pro plan.
Network	Private nodes, egress via Cloud NAT. Cloudflare (Full strict) → Ingress with a static IP.
Tools	Sublime Merge (git), k9s (cluster), GitHub web (PRs and review).
Out of scope	Prometheus, Grafana, e-mail alerts, arrivals, NetworkPolicy.

2. Architecture

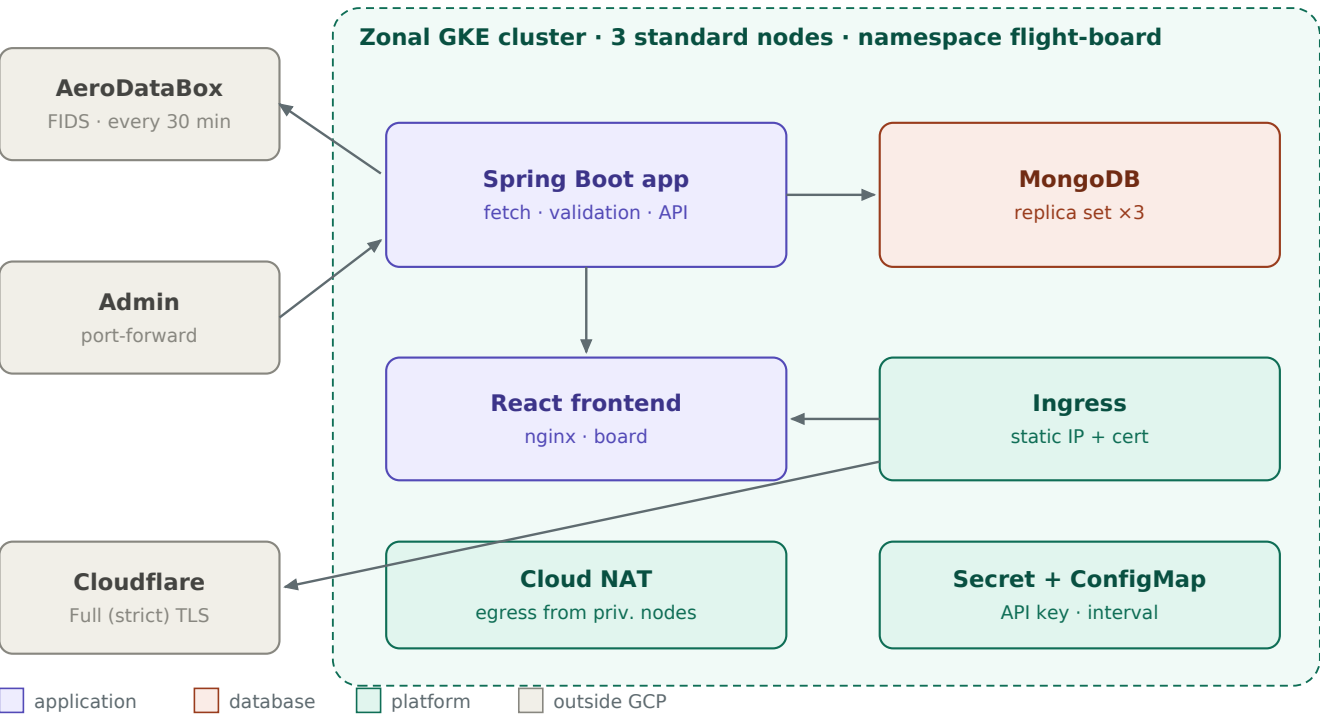


Fig. 1: The board never calls AeroDataBox directly; it only reads its own database through the API. Admin operations are not reachable through the Ingress.

3. Stage 0: setting up the agents and the cloud

Before the first line of code, the environment the agents work in is set up. Each agent has its own working directory (git worktree) so they do not switch branches under each other in a single directory.

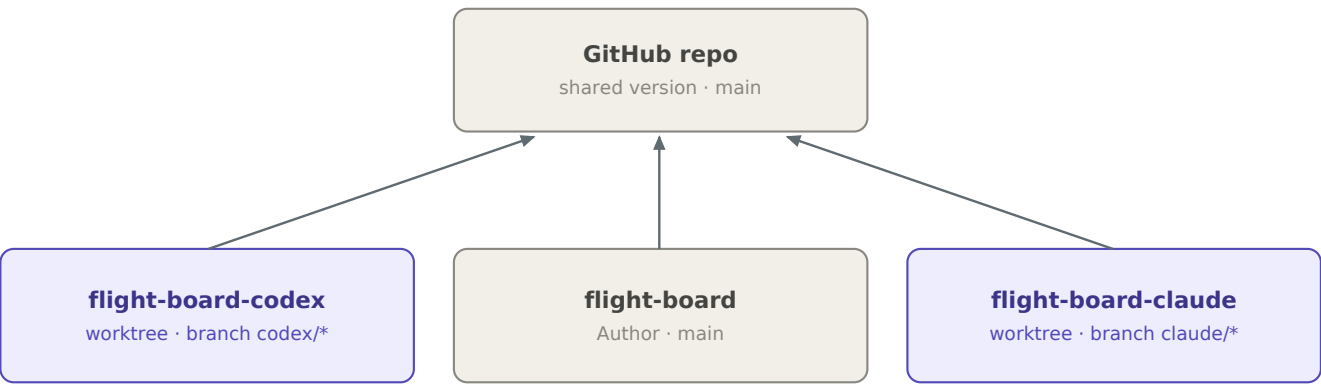


Fig. 2: One repository, three working directories. Each agent works only in its own.

What to set up	What the author needs to understand
One GitHub repository, branch protection on main	Where the shared version lives and why nobody writes to main directly
git worktree for each agent	Why two agents must not work in the same directory
Agent sign-in and permissions	Which account the agent works under and what it may run (gh, git, terraform plan, not apply)
AGENTS.md and CLAUDE.md	How the agent receives the rules and the project context
Issue → branch → PR → review → merge	How a task becomes an approved change
Shared app ↔ infra agreement (docs/contract.md)	Ports, environment variables, image names, health endpoints

First cloud setup (bootstrap)

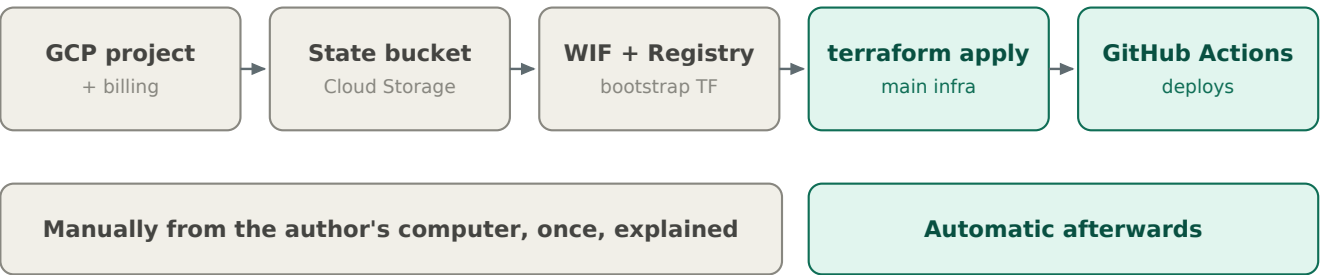


Fig. 3: GitHub Actions can deploy only after the permissions that allow it exist.

- Create the GCP project and link billing (manually in the console).
- A small separate Terraform in infra/bootstrap with local state: state bucket, Workload Identity Federation for GitHub and Artifact Registry. These resources are persistent and the main terraform destroy does not delete them. The author runs it once from their own computer.
- Once the bucket exists, the bootstrap state is migrated into it (terraform init -migrate-state, separate prefix). The local state file then does not remain on disk and never goes to git.

- Only then the main Terraform (infra/terraform) with state in the bucket, and GitHub Actions with keyless sign-in.

Who runs it	Infrastructure (Terraform)	Application (Helm)
On PR	GitHub Actions: fmt, validate, plan	GitHub Actions: tests, build
After merge	The author runs terraform apply after the plan is explained and reviewed	GitHub Actions: helm upgrade + verification; for infra changes only after a successful apply (A2)
MongoDB, TLS certificate	—	The author, once, when restoring the environment (ch. 15); certificate = Cloudflare Origin, inserted manually

4. How the agents guide the author

The goal is not only to build the project but for the author to understand it. For every task the agent works in three phases:

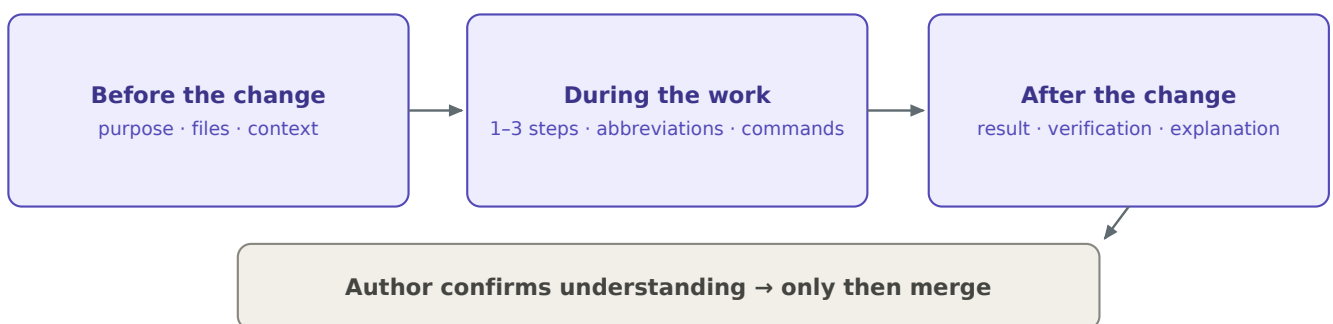


Fig. 4: Procedure for every task. A review by the other agent does not replace the author's understanding.

Phase	What the agent does
Before the change	Explains the purpose, which files will change and how they relate to the rest of the project. Waits for approval.
During the work	Works in 1-3 steps. Writes abbreviations with the full name in parentheses. Commands on one line with absolute paths. When editing a file, gives the complete content, not fragments.
After the change	Shows the result, how to verify it (command, k9s, URL) and explains the essential parts of the change.

Communication rules (part of CLAUDE.md and AGENTS.md)

- Communication with the author is in Slovak; established English IT terms are not translated (secrets, deployment, pod). All files in the repository (code, comments, docs, commit messages, PRs) are in English.
- Short sections, one thing at a time, ending with a question whether it is clear.
- Explain first, change afterwards. Approval applies to the explained task within the agreed scope; a change of scope needs new approval. Merges are always done by the author.
- Before every new task the agent updates its branch from main (git fetch + rebase).
- Links (URLs) in a separate block so they can be copied.
- Every PR has a section **Explanation for the author**: what the component does, how it is configured, where it gets its data, where it sends it and how to verify that it works.

The author does not need to know every line of Java code. The author must know what each component does, how it is configured, where it gets its data, where it sends it and how to verify that it works.

5. Infrastructure (Terraform)

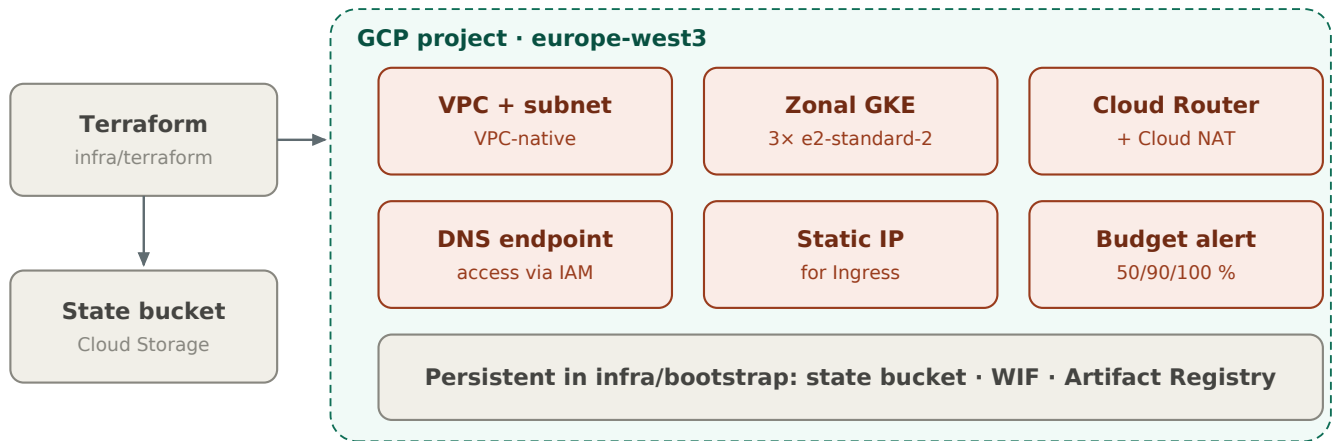


Fig. 5: GCP resources. The environment can be deleted (terraform destroy) and recreated; the state bucket remains.

- **Node pool:** 3 standard nodes from the start. Spot is not used, because switching spot ↔ standard on an existing pool is not possible and Terraform would delete and recreate the pool, i.e. all nodes at once.
- Node boot disks 30 GB (instead of 100 GB) to save budget.
- **Workload Identity Federation** and **Artifact Registry** (created in the bootstrap step): GitHub Actions signs in to GCP without a JSON key. The control plane **DNS endpoint** with IAM provides network access for GitHub Actions to the cluster.
- The **budget alert** at 50/90/100 % only sends e-mail; it does not stop billing. After work, terraform destroy can be run.
- MongoDB data is not part of Terraform. After destroy + apply the board is filled by a new fetch. Note: terraform destroy does not delete the MongoDB disks – the cleanup procedure is in chapter 15.

6. Placement in Kubernetes

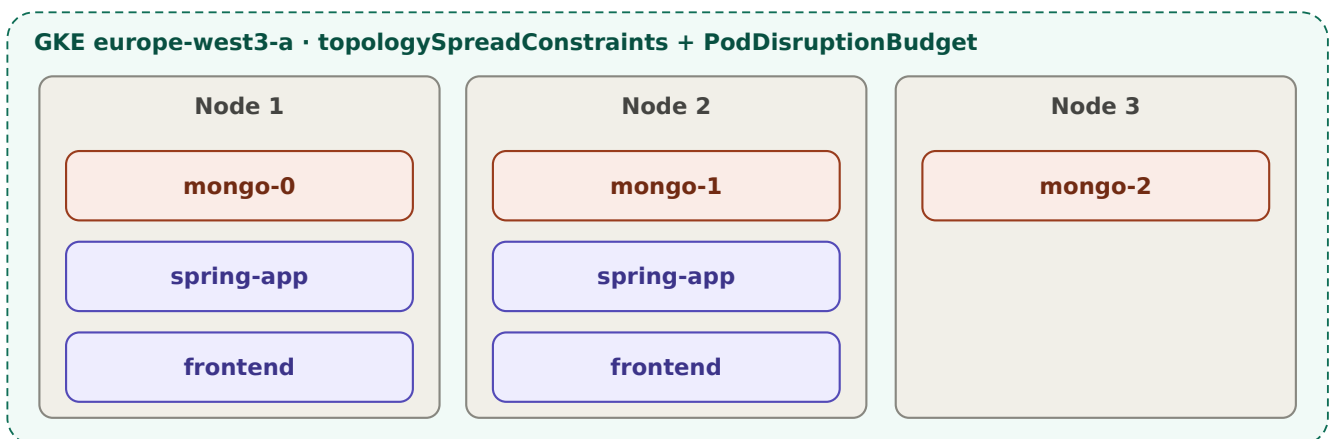


Fig. 6: Each replica on a different node. Draining one node does not stop the board.

Component	Replicas	PDB minAvailable	Placement
MongoDB (data members)	3	2	pod anti-affinity, 1 member per node, PVC per member
Spring Boot app	2	1	topologySpreadConstraints by hostname

Component	Replicas	PDB minAvailable	Placement
Frontend (nginx)	2	1	topologySpreadConstraints by hostname

- A PDB (PodDisruptionBudget) protects during a planned drain, not during a sudden machine failure.
- Electing a new MongoDB primary may cause a short interruption. Meanwhile the frontend shows the last board.
- **Liveness** = the application is alive. **Readiness** = it can serve requests (database reachable). An AeroDataBox outage must affect neither, so it never causes a pod restart.

7. Fetching data

Coordination between replicas

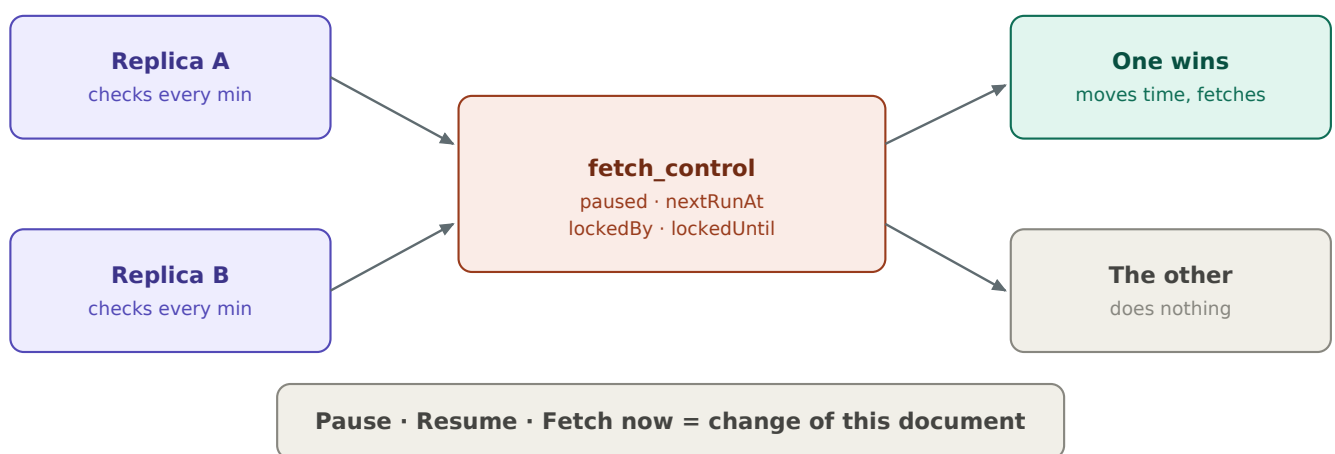


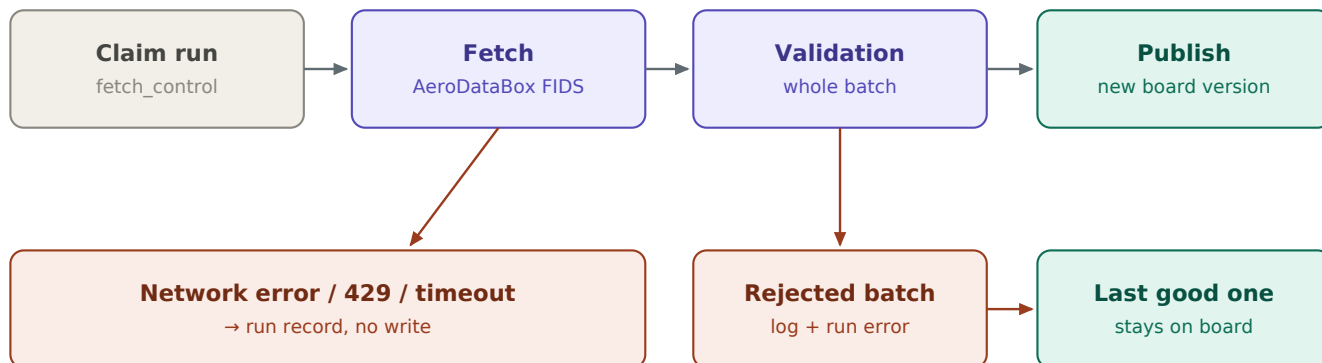
Fig. 7: Both replicas try to claim a run every minute. Only one ever succeeds.

- Control document `fetch_control` in MongoDB: `paused`, `nextRunAt`, `lockedBy`, `lockedUntil`.
- Claiming a run is one atomic `findOneAndUpdate` operation with the condition: not paused, `nextRunAt ≤ now` and the lock is free or expired. The same operation sets `nextRunAt = now + interval` and the lock.
- This makes two runs right after each other impossible, and the quota is used only once per interval (30 min $\approx$ 2,880 units per month).
- State lives in the database: a pause applies to both replicas and survives a pod restart.
- The `fetch_control` document is created at startup if it does not exist: `paused = false`, `nextRunAt = now`, free lock, `fetchSeq = 0`. A restart does not reset an existing pause, lock or counter.
- The lock is valid for 5 minutes (`lockedUntil`). A run has a two-minute limit. It may publish only when the transaction conditions are met, including a valid lock. A failed run releases its own lock if the database is reachable; the last good board stays unchanged.
- Each run has a unique `runId` (UUID) stored in `lockedBy`. Releasing the lock is conditional: `lockedBy = my runId`. A replica never releases someone else's lock.
- When claiming a run, the run receives `fetchSeq` (an increasing number, incremented in the same atomic operation).
- Publishing is **one MongoDB transaction**, not “read, compare, write”: (1) a conditional write to `fetch_control` filtered on `lockedBy = my runId` and `lockedUntil > now`, which also releases the lock; (2) a write to `board_current`: if the board does not exist it is created (fixed `_id`); if it exists, only a run with a higher `fetchSeq` may replace it. If any condition fails, the transaction is

aborted and the board stays unchanged. A stuck or older run therefore never overwrites a newer result.

- After a replica crash, the system waits both for the lock to expire and for nextRunAt. Every manual Fetch now uses quota (2 units).

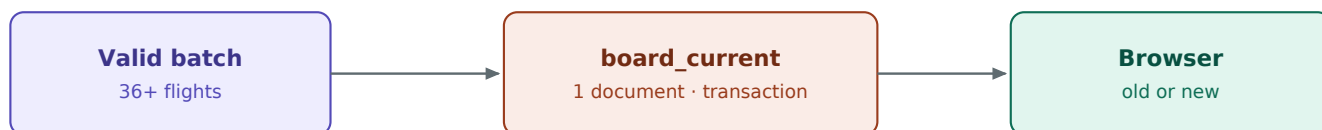
Batch processing



A valid empty response $\neq$ an error response. Flights that disappear from a new batch are removed from the board.

Fig. 8: Only a complete, validated batch is published. On error the last good version remains.

Atomic board publishing



Replacing a single document is atomic: never a mix of old and new flights.

- The current board is **one document** board_current (flights, publish time, run ID). On publish it is replaced entirely at once (within the transaction above). With ~100 flights the document is a few tens of kB.
- Every run is recorded in fetch_runs with its run ID: time, result, HTTP code, number of flights, error. The run ID also appears in every log line.
- The **interval** is a ConfigMap value (Helm values). Changing it = editing a number and helm upgrade; the code does not change. The chart puts a ConfigMap checksum annotation on the pod, so a configuration change automatically replaces the pods.
- Retention: fetch_runs has a 7-day TTL index. A board_current older than 12 hours is not returned by the backend and is deleted by a daily job. Only synthetic test data goes into the repository.

8. Admin operations

Operation	What it does
Pause fetching	Sets paused = true. No replica starts a new run; a run already in progress finishes.
Resume fetching	Sets paused = false. The next run follows nextRunAt.

Operation	What it does
Fetch now	One atomic operation: if not paused and no valid lock, sets nextRunAt = now and the run starts at the next one-minute check. While paused or locked it is rejected with an explanation.

The /admin/* endpoints run on a separate port that listens only on **127.0.0.1** inside the pod and is not in any Service or Ingress. It is not reachable from another pod or from the internet. kubectl port-forward connects to the pod's localhost, so it works (in k9s with Shift+F). Pause stops new runs; a run already in progress finishes.

9. The board

Visual template

DEPARTURES FROM TERMINAL 2																			15:28	
Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark
14:55		ZURICH	LX1479	284 - 295	272 - 274	Cancelled	18:00		BUDAPEST	FR4113	208		Check-in	19:20		AMSTERDAM	KL1360	284 - 295	260 - 262	Opens at 18:40
15:15		BARCELONA	VY8651		260 - 262	Gate closed	18:05		WARSAW	LO522			Check-in	19:20		KOS	OK4402	284 - 295		Opens at 18:45
15:35		MADRID	IB3127		284 - 295	Boarding	18:20		PALMA DE MALLORCA	OK4376	284 - 295		Check-in	19:30		HELSINKI	AY1224	284 - 295		Opens at 19:00
15:45		BARCELONA	FR6378	210		Gate open	18:20		RHODES	QS1284			Check-in	19:30		LISBON	TP1243			Opens at 19:05
16:20		RIMINI	FR4826		272 - 274	Check-in	18:30		MILAN/BERGAMO	FR3218			Check-in	19:45		WARSAW	LO0526			Opens at 19:05
16:45		VIENNA	OS711	284 - 295	272 - 274	Check-in	18:30		BOLOGNA	FR4721	284 - 295	260 - 262	Check-in	19:55		PARIS/CDG	UJ8083	284 - 295	260 - 262	Opens at 19:05
16:55		MUNICH	LH1691		200 - 201	Check-in	18:30		AMSTERDAM	KL1358	284 - 295		Check-in	20:25		DUSSELDORF	OS8128			Opens at 19:05
17:10		PALMA DE MALLORCA	EW5892		260 - 262	Check-in	18:35		CORFU	OA3421	284 - 295	272 - 274	Opens at 16:50	20:40		MALTA	KM0539			Opens at 19:05
17:20		RHODES	FR7364	284 - 295	272 - 274	Check-in	18:40		FRANKFURT	LH1401			Opens at 17:00	20:45		PARIS/ORY	T04395			Opens at 19:05
17:30		MALAGA	FR3058	209		Check-in	18:45		ATHENS	A3607	284 - 295	272 - 274	Opens at 17:30	20:55		MARSEILLE	FR8228			Opens at 19:05
17:35		GOTHENBURG	SK1786	284 - 295	260 - 262	Check-in	18:50		BRUSSELS	SN2816	284 - 295	272 - 274	Opens at 17:45	21:00		COPENHAGEN	SK0768	284 - 295		Opens at 19:05
17:35		PARIS/CDG	AF1583		272 - 274	Check-in	19:10		ZURICH	LX1487	284 - 295	272 - 274	Opens at 18:25	21:05		STOCKHOLM	SK1768	284 - 295		Opens at 19:05
DEMO - SAMPLE DATA																				

DEMO - SAMPLE DATA

Fig. 9: Template. Values in the image are samples.

- English. Light-blue header with title and clock, a row of column names, rows alternating dark grey and blue.
- Three blocks of 12 rows side by side (36 flights), sorted by scheduled time. Departed flights are hidden.
- Condensed font (Barlow Condensed or Roboto Condensed). Remark: Cancelled red, Boarding yellow, others white.
- Bottom right a “data: AeroDataBox” link (working link to aerodatabox.com). No airport or airline logos.

Column-to-data mapping

Column	Source (AeroDataBox)	Rule
Sched.	scheduled local time	HH:MM
Exp.	revised time	only if different from scheduled, otherwise empty
Destination	arrival airport	city, upper case
Flight	flight number	no spaces (LX1479)
Check-in	checkInDesk	field exists in the schema, may be empty
Bag Drop	not in the schema	empty column

Column	Source (AeroDataBox)	Rule
Remark	flight status	Boarding, Gate closed, Departed, Cancelled, Delayed; "Opens at" is not computed
Terminal 2	terminal field	if the source does not provide the terminal, title "DEPARTURES" and all departures are shown

All fields must be verified against the first real response from the source. Missing data stays empty; nothing is invented.

Flight selection

- Request: FIDS for PRG, direction **Departure**, window from now –1 h to now +11 h (12 h, the maximum for the Pro plan), without cargo, without codeshare (the source's filter is partly estimated, A5).
- All times in the **Europe/Prague** time zone. Sorting by full date and time, not HH:MM only (flights after midnight).
- Terminal 2 filter (if the source provides it), hide departed flights. The first 36 flights are shown, the rest are not.

Status from source	Remark on the board
Boarding	Boarding (yellow)
GateClosed	Gate closed
Delayed	Delayed
Canceled	Cancelled (red)
CanceledUncertain	empty – an uncertain state must not be shown as a cancellation
Departed	the flight is hidden
other / unknown	empty + log entry

Board states

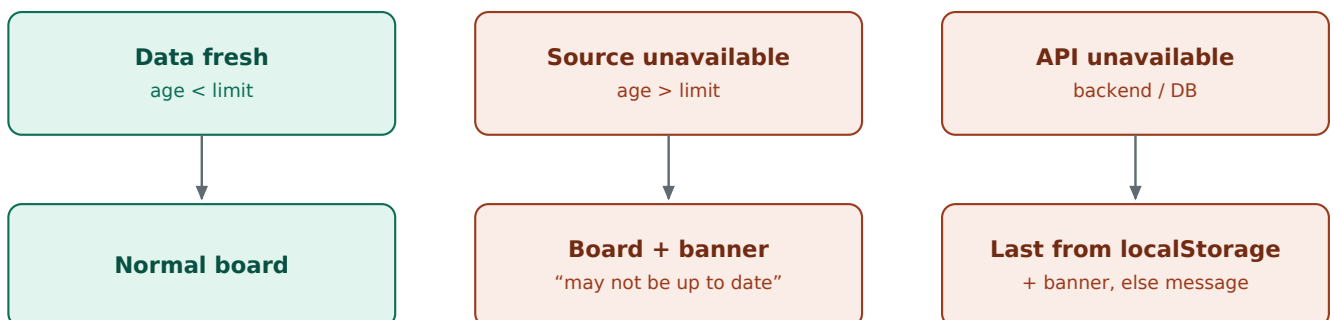


Fig. 10: The board never shows erroneous data and always says why data may not be up to date.

- Data age is computed by the backend from the last successful publish. The "stale data" limit is in the ConfigMap (e.g. 75 min with a 30-min interval).
- The frontend stores the last board in localStorage together with the publish time. During an outage the data age keeps being computed from that time. A copy older than 12 hours is deleted and not shown.
- On a first visit during an outage there is nothing to show: message "Flight information is temporarily unavailable".

Refresh

Activity	Interval	Note
Check whether it is time to fetch	1 min	every replica; fetch_control decides
Fetch from AeroDataBox	30 min	ConfigMap, uses quota
Load the board from own API	30 s	browser, no quota used
On-screen clock	1 s	browser only

10. API

Endpoint	Port	Content
GET /api/departures	public	flights + data age + last publish time
GET /api/status	public	last successful run, last error, data age, paused, version (git SHA)
GET /actuator/health/liveness	internal	application alive
GET /actuator/health/readiness	internal	database reachable
POST /admin/pause resume fetch-now	127.0.0.1	only via port-forward

11. Working with two agents

The agents work as a small team: each owns its part of the repository, they review each other's work, and the author acts as tech lead. Nobody writes directly to main.

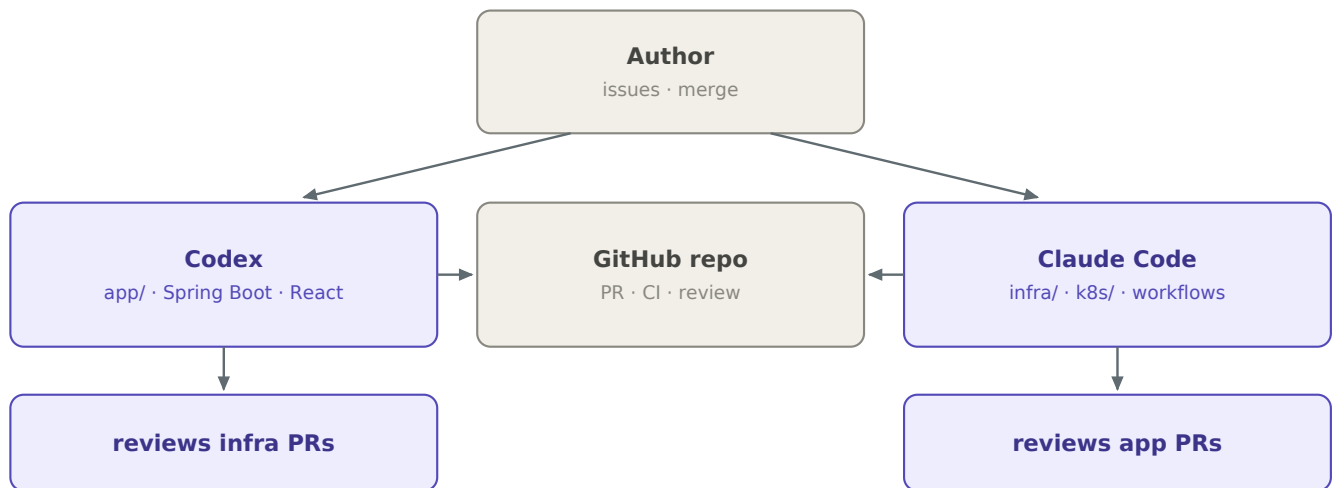
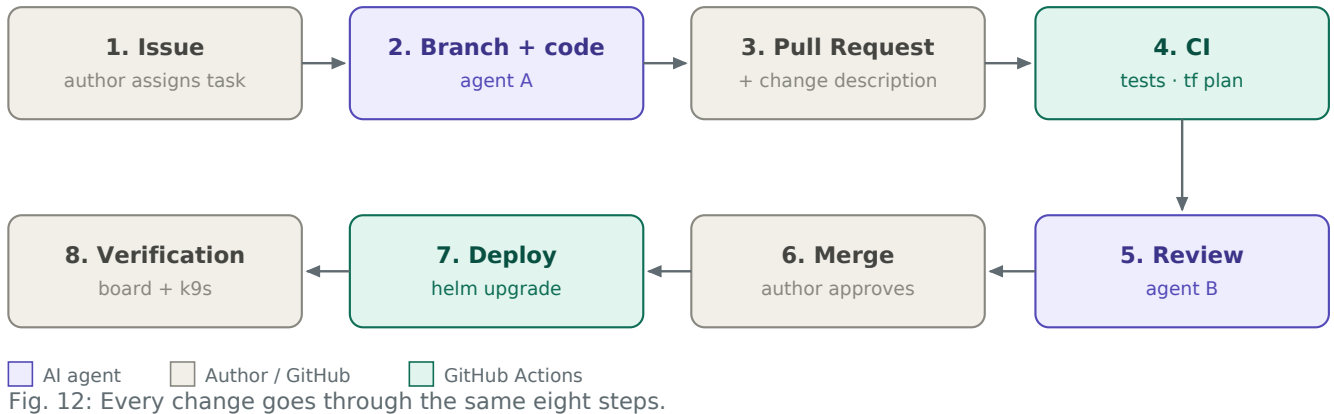


Fig. 11: Role split. The agents do not change each other's directories, so no conflicts arise.

	Codex	Claude Code
Owns	app/backend (Spring Boot), app/frontend (React + TS), tests, Dockerfiles in app/	infra/, k8s/ (Helm chart), .github/workflows, docker-compose.yml, docs/
Branch prefix	codex/<issue>-description	claude/<issue>-description
Worktree	flight-board-codex	flight-board-claude
Rules	AGENTS.md in the repo root	CLAUDE.md in the repo root
Reviews	PRs from Claude Code (infra, CI/CD)	PRs from Codex (application)

	Codex	Claude Code
Must not change	infra/, k8s/, .github/, docker-compose.yml	app/ (only comments in review)

Lifecycle of a single change



Contents of CLAUDE.md and AGENTS.md

- Project goal and architecture in 5 sentences, reference to this document.
- Owned directories and what the agent must not change.
- Rules: no secrets in code, one task = one PR, PR with a “what, why, how to test” description and an “Explanation for the author” section.
- Communication rules and the before / during / after procedure from chapter 4.
- How to run the tests and the local setup (Docker Compose).
- In review, look for bugs and risks, not style. The review result is a comment on the PR.

Assigning tasks

- One task = one GitHub issue with a verifiable result.
- The agent creates a branch with its prefix in its worktree and opens a PR. The other agent receives the PR link for review.
- The author looks at the diff in Sublime Merge, including the list of changed files, and at the CI result on GitHub. The author merges; what the author cannot explain is not merged.
- Directory ownership is an agreement, so CI also enforces it (chapter 13).

12. Repository structure

```

flight-board/
├── AGENTS.md           rules for Codex
├── CLAUDE.md          rules for Claude Code
├── docker-compose.yml  local setup (Claude Code)
├── app/
│   ├── backend/       Spring Boot (fetch, validation, API, admin)
│   └── frontend/      React + TypeScript, nginx
├── infra/bootstrap/    state bucket, WIF, Artifact Registry (persistent)
├── infra/terraform/    VPC, GKE, NAT, DNS endpoint, IP, budget
├── k8s/flight-board/   Helm chart (app, frontend, values, PDB)
├── .github/workflows/  pr.yml (tests + plan), deploy.yml (build + helm)
└── docs/              this document, contract.md, runbook, cleanup
  
```

13. CI/CD

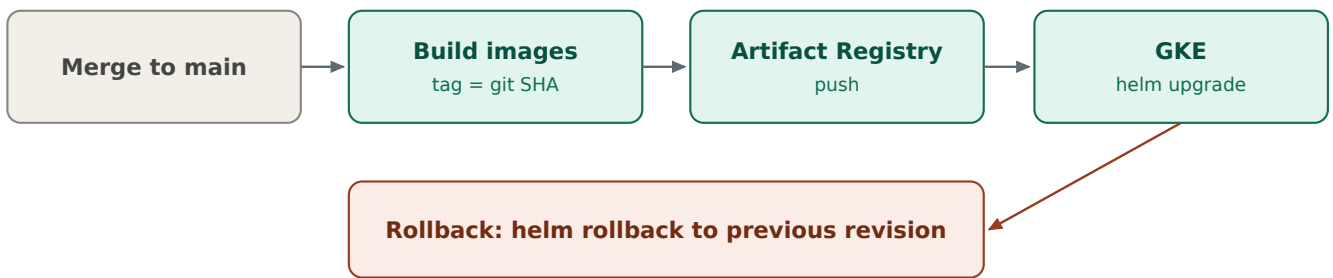


Fig. 13: After merge, the version tagged with the commit is deployed. Returning to the previous version is one command.

- **Pull Request:** mvnw verify, npm test, tsc --noEmit, npm run build, terraform fmt / validate / plan. No deployment.
- **Ownership check:** a PR from a codex/* branch fails if it changes infra/, k8s/, .github/ or docker-compose.yml; a PR from claude/* fails if it changes app/.
- **Merge to main:** build and push images (tag = git SHA), helm upgrade, then verify readiness and the git SHA in /api/status. For infrastructure changes only after a successful manual apply (A2). If verification fails, the deploy is red.
- The MongoDB replica set is installed once with a separate Helm chart, not on every application deploy.

14. Testing

- Unit tests of batch validation: valid, valid empty, corrupted, incomplete response.
- Coordination tests: two concurrent claims → only one succeeds; a pause blocks both replicas; an expired lock after a crash can be claimed; Fetch now during a pause is rejected; an older run does not overwrite a newer result; a run with an expired lock does not publish; someone else's lock is not released.
- Mapping tests: Canceled → Cancelled, CanceledUncertain → empty, sorting across midnight, 36-flight limit.
- Integration tests against MongoDB (Testcontainers) and against a test source with synthetic responses (200, 429, 500, timeout, malformed JSON). The real AeroDataBox quota is never used in tests.

15. Cleaning up the environment

Deleting the GKE cluster does not delete the disks Kubernetes created for MongoDB (PVCs). They keep being billed. Cleanup therefore happens in this order:

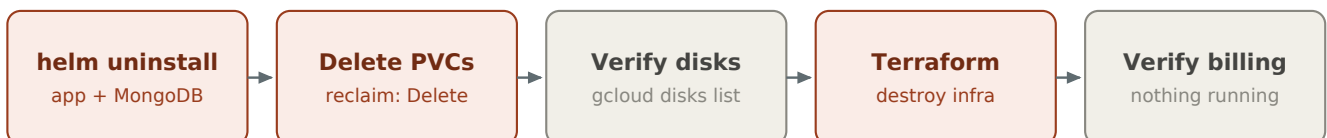


Fig. 14: Application and storage first, infrastructure afterwards. The procedure is also in docs/ as a script with checks.

- After helm uninstall and deleting the PVCs, **wait** until the disks (gcloud compute disks list) and the Ingress load balancer (gcloud compute forwarding-rules list) are gone. Only then terraform destroy.

- Kept on purpose (infra/bootstrap): state bucket, Workload Identity Federation, Artifact Registry with images; the GCP project. Everything else must be gone.

Restoring the environment from scratch

Step	What	Who
1	terraform apply (infra/terraform) after reviewing the plan	Author
2	gcloud container clusters get-credentials with the DNS endpoint (Appendix A8), check nodes in k9s	Author
3	namespace, Secret with the AeroDataBox API key and MongoDB passwords	Author (from password manager)
4	helm install MongoDB replica set, verify 3 members and a primary	Author
5	Cloudflare Origin certificate as a TLS Secret, check the DNS record points to the static IP	Author
6	Run the deploy workflow (helm install of the application)	GitHub Actions
7	Fetch now, verify /api/status and the board over HTTPS	Author

The procedure is in docs/runbook.md with exact commands and the expected result of each step.

Appendix A: Implementation rules

Rules the agents must follow so that the document leads to the same solution. Where a brief description in a main chapter differs from an implementation rule in Appendix A, Appendix A prevails. The rules are carried over into docs/contract.md and into the individual issues.

A1. Transactions in the application

- The Spring application has a MongoDB transaction manager configured (or uses transactions through the driver API).
- If a conditional write does not modify the expected document (0 modified), the application explicitly aborts the transaction. Zero modified documents is not a database exception.
- The AeroDataBox call is outside the transaction. When a transaction is retried, the already downloaded batch is processed; the source is not called again.
- Transient transaction errors and an unknown commit result are handled with retries and a retry limit.

A2. Deployment and Terraform

- For changes to the main infrastructure, the application deployment starts only after a successful manual terraform apply. An application-only change is deployed automatically after merge.
- The deploy workflow supports manual triggering (workflow_dispatch) for environment restoration.
- Deployments run one at a time (concurrency group per environment). Before deploying, check that the commit is not older than the deployed one.
- GitHub Actions uses the cluster DNS endpoint (explicitly enabled in the get-credentials step).

A3. Agents and repository

- The root files AGENTS.md, CLAUDE.md, README.md and .gitignore are created and maintained by Claude Code; the author reviews them, Codex comments.
- Stage 0 defines the computer, shell and actual directories of both agents. Worktrees are used on one computer with a shared, accessible Git repository; with separate machines, separate clones are used.
- Required checks on main are enabled only after the first working check workflow exists.
- The ownership check also rejects an unknown branch prefix. The prefix identifies the task, it does not verify the agent's identity.

A4. docs/contract.md - decisions before implementation

Area	What exactly to define
Network	Standard external GKE Ingress; /api/* → backend, / → frontend. Admin and Actuator are not routed publicly.
IP address	Global static IP for the Ingress, its name in the configuration.
Ports and health	Exact ports of the application, admin and probes. Readiness checks MongoDB, liveness does not. Probes must not use the admin port (127.0.0.1).
Load balancer	Own health checks for backend and frontend via BackendConfig.
Permissions	Separate accounts for terraform plan, application deploy and nodes (reading from Artifact Registry). Federation restricted to the repository and allowed workflows / branches.
Versions and storage	Specific MongoDB chart, version, image; 3 data members, CPU/RAM requests, disk size, reclaim policy.
Configuration	Names of variables, secrets, images, database and replica set. When the interval changes: the stored nextRunAt is not recomputed; the new interval applies from the next run.

A5. Source data

- Request with withLeg=false. For departures, the data in movement describes the departure from Prague, but movement.airport is the destination airport. Exact paths to time, terminal and destination are in contract.md.
- Terminal 2 mode shows only flights with a confirmed terminal 2. Flights without a terminal are not guessed.
- The source's codeshare filter is partly estimated; duplicates cannot be fully ruled out.

A6. Data states and API

State	/api/departures response	Text on the board
Valid batch with flights	200, flights + publishedAt	—
Valid empty batch	200, empty list + publishedAt	No departures in the next hours
Data older than the limit	200, flights + stale = true	Information may not be up to date
No valid board (new DB, expired)	503, code NO_DATA	Flight information is temporarily unavailable
API unavailable	—	last copy + Connection lost; without a copy: unavailable

- A valid empty batch is a success and replaces the old flights.
- Data age = time since the successful fetch and publish, not a guarantee that the provider's data is current.
- The browser cache stores the original publishedAt (does not refresh it). It is checked on page open and while the page runs; an older delayed response does not overwrite a newer copy. Deletion in a closed browser is not promised.
- Fewer than 36 flights: the remaining rows stay empty. Minimum screen size 1280 × 720; below it the blocks wrap underneath each other.
- The image is a template for layout and colours; its sample Bag Drop and “Opens at” values are not carried into the data.
- /api/* sends Cache-Control: no-store so that neither nginx nor Cloudflare hides the data age.

- The public `/api/status` returns a short error without the API key, passwords or exception stack trace.

A7. Checks and tests

- Backend: `mvnw verify` (unit and integration tests). Frontend: `npm test`, `tsc --noEmit` and `npm run build`.
- Additional tests: first publish into an empty DB, publish after an expired board was deleted, concurrent initialization of `fetch_control`, rejection of `Fetch` now with an active lock and during a pause.
- Post-deploy verification checks that `/api/status` returns the expected git SHA, not just HTTP 200.

A8. Cleanup and restore

- The cleanup script deletes only resources of this project (filter by label / name).
- Besides forwarding rules, the other load balancer parts are verified too (backend services, health checks, NEG's, URL maps, target proxies). Dependent network resources are cleaned up before the cluster is deleted.
- Restore: `gcloud container clusters get-credentials <cluster> --zone europe-west3-a --dns-endpoint`.
- The Cloudflare DNS record must stay proxied (orange cloud). The Origin certificate is not trusted for direct browser access.